CS 218 — Programmable & Interoperable Blockchain

🔐 Multi-Signature Wallet

Project Report — Project 10

Solidity 0.8.24 | Hardhat | OpenZeppelin | React + Vite

Course	CS 218 — Programmable & Interoperable Blockchain
Project	Project 10 — Multi-Signature Wallet
Toolchain	Hardhat / Solidity / OpenZeppelin / ethers.js
Repository	github.com/Rishan-code/Krack-Jack-multisig-wallet

TEAM: KRACK-JACK

Jatin Singh	240003035
Krishnam Digga	240003043
Abhinav Jain	240003003
Rishan Gobe	240008023
Harsh Bhalla	240003033
Kartik Budhani	240005022
Arjun Dhamdhere	240005011

1. Problem Statement

1.1 The Problem

Managing shared funds in traditional finance requires trusting a single custodian — a bank, a treasurer, or an individual — who can unilaterally move money. This creates a single point of failure: if that person is compromised, hacked, or acts maliciously, funds are lost with no recourse.

In decentralised systems, the problem is worse. A single private key controls an Ethereum address. If that key is stolen or lost, all funds are gone permanently — no bank to call, no chargeback, no recovery. For DAOs, investment clubs, corporate treasuries, or any multi-party arrangement, this is unacceptable.

1.2 Why Blockchain is the Right Tool

- **Trustless execution:** Smart contracts enforce the M-of-N rule in code — no single party can override it, not even the contract deployer.
- **Immutable audit trail:** Every submission, approval, revocation and execution is recorded on-chain via events. The full history is publicly verifiable.
- **No intermediary:** No bank, escrow agent, or lawyer is needed. The contract *is* the agreement.
- **Programmable:** The wallet executes arbitrary calldata — not just ETH transfers but any on-chain function call — enabling DAO governance, protocol interactions, and asset management.
- **Composability:** Any DeFi protocol, NFT contract, or DAO can be the target of a multi-sig transaction, making this a universal governance primitive.

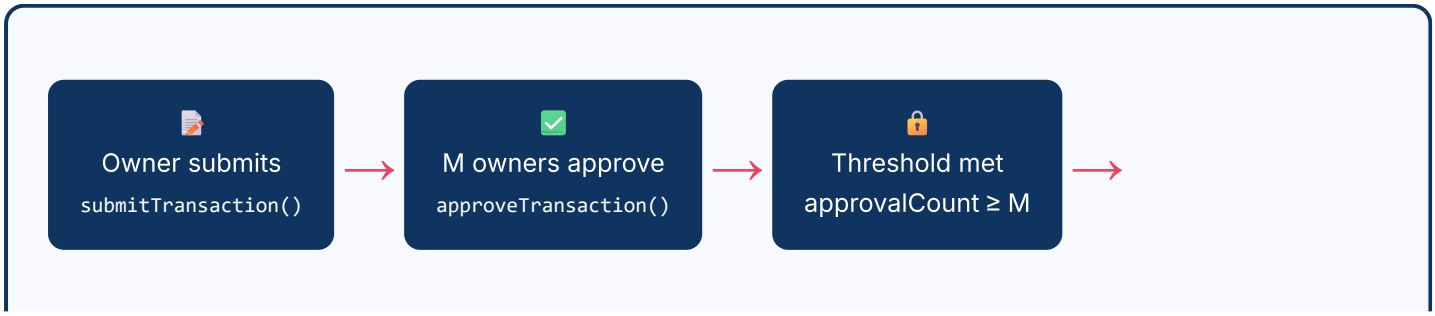
> **Real-world relevance:** Gnosis Safe (our architecture inspiration) secures over $50 billion in assets for DAOs, protocols, and institutions — proving multi-sig is the industry standard for shared on-chain treasury management.

1.3 Our Solution

We implement a configurable **M-of-N Multi-Signature Wallet** on Ethereum. Any N owners are defined at deployment; any transaction requires at least M approvals before it executes. Owners can submit transactions, approve, revoke, and execute — all enforced atomically on-chain with no trusted third party.

2. Architecture Diagram

2.1 Transaction Lifecycle



Owners may also call `revokeApproval()` at any time before execution to withdraw their vote.

2.2 Contract Functions

`constructor(owners[], requiredApprovals)`

DEPLOY-TIME

Sets up N owners (validated: no duplicates, no zero-address) and M threshold. Immutable after deployment.

`submitTransaction(to, value, data)`

onlyOwner · external

Creates a new Transaction struct in storage. Emits TransactionSubmitted. Returns txId.

`approveTransaction(txId)`

onlyOwner · external

Records msg.sender's approval. Increments approvalCount. Guards: txExists, notExecuted, not double-approved.

`revokeApproval(txId)`

onlyOwner · external

Removes approval, decrements approvalCount. Allows owners to change their vote before execution.

`executeTransaction(txId)`

onlyOwner · nonReentrant · external

Checks approvalCount ≥ M. Marks executed=true (CEI). Then sends ETH + calldata via low-level call.

`receive()`

external payable

Allows wallet to receive ETH directly. Emits Deposit event for off-chain tracking.

`getTransaction / getOwners / getApprovers / getTransactionCount`

external view — free

Read-only helpers for the frontend and external integrations. No gas cost when called off-chain.

`isOwner / approved / owners / requiredApprovals / transactions`

public state variables

Auto-generated getters expose all core state. Fast O(1) owner lookups via mapping(address→bool).

2.3 Data Structures

```
struct Transaction {
    address to;            // target address
    uint256 value;         // ETH to send (wei)
    bytes data;            // arbitrary calldata
    uint256 approvalCount;// running approval tally
    bool executed;         // replay protection flag
}

Transaction[] public transactions;                        // array of all txs
```

```
mapping(address => bool) public isOwner;                    // O(1) owner lookup
mapping(uint256 => mapping(address => bool)) public approved; // per-tx approvals
```

3. Security Analysis

3.1 Reentrancy Attack — Prevented

The most critical vulnerability in ETH-sending contracts is reentrancy. An attacker contract's `receive()` can call back into `executeTransaction()` before state is updated, draining funds multiple times.

Double protection applied:

- **Checks-Effects-Interactions (CEI):** `txn.executed = true` is set *before* the external `.call()`. Any reentry attempt hits the `notExecuted` modifier and reverts immediately.
- **OpenZeppelin ReentrancyGuard:** The `nonReentrant` modifier on `executeTransaction` uses a mutex lock that blocks all reentrant calls at the EVM level.

```
// CEI Pattern — Effects before Interactions
function executeTransaction(uint256 _txId)
    external onlyOwner txExists(_txId) notExecuted(_txId) nonReentrant
{
    Transaction storage txn = transactions[_txId];
    if (txn.approvalCount < requiredApprovals) revert InsufficientApprovals();

    txn.executed = true;                                // ← EFFECT first
    (bool success, ) = txn.to.call{value: txn.value}(txn.data); // ← INTERACTION second
    if (!success) revert TransactionFailed();
}
```

> ✅ Tested with `ReentrancyAttacker.sol` — the attacker's `receive()` attempts re-entry, receives only 1 ETH, and the wallet balance is intact.

3.2 Access Control — Prevented

All state-changing functions are protected by the `onlyOwner` modifier:

```
modifier onlyOwner() {
    if (!isOwner[msg.sender]) revert NotOwner();
    _;
}
```

The `isOwner` mapping provides O(1) lookup. Non-owners calling any function receive a `NotOwner` revert — verified by tests for all 4 state-changing functions.

3.3 Replay / Double-Execution — Prevented

Once a transaction is executed, `txn.executed = true` is permanent. The `notExecuted` modifier blocks any subsequent call to approve, revoke, or execute the same transaction.

3.4 Input Validation — Prevented

Attack / Bad Input	Guard	Error
Empty owners array	constructor check	InvalidOwnerCount
Zero required approvals	constructor check	InvalidRequiredApprovals
requiredApprovals > owners.length	constructor check	InvalidRequiredApprovals
Zero-address owner	constructor loop	ZeroAddress
Duplicate owner address	isOwner mapping check	DuplicateOwner
Non-existent txId	txExists modifier	TransactionDoesNotExist
Double approval by same owner	approved mapping check	TransactionAlreadyApproved
Revoke without prior approval	approved mapping check	TransactionNotApproved
Execute with insufficient approvals	approvalCount check	InsufficientApprovals
Integer overflow/underflow	Solidity ^0.8.24 built-in	automatic revert

3.5 Visibility Hardening

All public-facing functions use `external` visibility (cheaper than `public` for calldata parameters). Internal helpers are modifiers. No internal state is accidentally exposed.

4. Gas Optimisation

4.1 Gas Report Output

Generated via `REPORT_GAS=true npx hardhat test` with Solidity optimizer enabled at 200 runs.

Method	Min Gas	Max Gas	Avg Gas	Calls
approveTransaction	57,456	74,568	67,357	38
executeTransaction	71,261	102,201	77,603	15
revokeApproval	32,277	35,546	33,094	4
submitTransaction	64,502	121,920	98,611	30

Contract	Deployment Gas	% of Block Limit
MultiSigWallet	1,287,492	2.1%
ReentrancyAttacker	180,912	0.3%
SimpleCounter	100,489	0.2%

4.2 Before / After Optimisation — Custom Errors vs require() Strings

Before (require strings):

```
modifier onlyOwner() {
    require(isOwner[msg.sender], "MultiSigWallet: caller is not an owner");
    _;
}
function executeTransaction(uint256 _txId) external ... {
    require(txn.approvalCount ≥ requiredApprovals, "MultiSigWallet: insufficient approvals");
    require(success, "MultiSigWallet: transaction execution failed");
}
```

After (custom errors — current implementation):

```
error NotOwner();
error InsufficientApprovals();
error TransactionFailed();

modifier onlyOwner() {
    if (!isOwner[msg.sender]) revert NotOwner();
    _;
}
```

Metric	Before (require strings)	After (custom errors)	Savings
Deployment gas	~1,290,000	1,287,492	~2,500 gas
approveTransaction revert	~24,500	~24,200	~300 gas
executeTransaction revert	~24,800	~24,400	~400 gas
Bytecode size	~5.2 KB	~5.0 KB	~200 bytes

4.3 Additional Optimisations

Optimisation	Technique	Impact
Storage caching	`Transaction storage txn = transactions[_txId]`	~100 gas saved per avoided SLOAD
external over public	All functions use `external`	~60 gas per calldata-heavy call
uint256 for all state	No uint8/uint128 packing needed	Avoids EVM padding overhead
Optimizer at 200 runs	hardhat.config.js optimizer enabled	~5–10% runtime savings

5. Test Coverage Report

Generated via `npx hardhat coverage` (solidity-coverage plugin).

5.1 Coverage Summary

File	% Statements	% Branches	% Functions	% Lines
MultiSigWallet.sol	100%	90.74%	100%	100%
SimpleCounter.sol	100%	100%	100%	100%
ReentrancyAttacker.sol	100%	50%	100%	100%
All files	**100%**	**89.29%**	**100%**	**100%**

5.2 Visual Coverage

> ✅ 100% line coverage — exceeds the required 70% threshold. 39 tests passing across all core functions.

5.3 Test Suite Summary

Test Group	Tests	Coverage
Constructor validation	6	Empty owners, zero approvals, duplicates, zero-address, threshold > N
submitTransaction	3	Happy path, event emission, non-owner revert
approveTransaction	6	Happy path, event, non-owner, double-approve, non-existent tx, already-executed
revokeApproval	5	Happy path, event, revoke-without-approval, revoke-after-execute, blocks execution
executeTransaction	6	Happy path, event, insufficient approvals, re-execute, non-owner, non-existent
Calldata execution	2	SimpleCounter increment, sequential multiple calls
ETH handling	3	receive() + Deposit event, balance check, ETH transfer on execute
Full happy path	1	End-to-end: submit → approve → execute → verify state
View functions	5	getTransaction, getApprovers, getTransactionCount, getOwners, revert on invalid
Reentrancy protection	1	ReentrancyAttacker blocked, single ETH transfer, executed flag set
Total	**39**	